

Centro de Enseñanza Técnica Industrial (CETI)
Plantel Colomos
Departamento de Sistemas y Computación

Proyecto Final – Sistemas Expertos

TAI – Asesor de Negocios IA

Simulador Económico de Cafetería
vía Bot de Discord con Motor de Inferencias

Alumno: Alvaro Martin Ortiz Ascencio

Materia: Sistemas Expertos

Profesor: Mauricio Alejandro Cabrera Arellano

Grupo: 7E

Fecha: Junio 2026

Repositorio: <https://github.com/Ortiz-Ascencio-Alvaro-Martin/SE-Discord>

Prototipo: <https://gown-kinder-24829009.figma.site/>

Invitar bot: https://discord.com/oauth2/authorize?client_id=1511180179395838083

Índice

1. Objetivo	2
2. Descripción del Sistema	2
3. Arquitectura del Sistema	3
4. Base de Conocimiento	3
5. Inferencias Utilizadas	4
6. Explicación de los Agentes	6
7. Capturas del Sistema	8
8. Base de Datos	8
9. Herramientas Utilizadas	9
10. Conclusiones	10
11. Manual de Usuario	11

Objetivo

Diseñar e implementar un **sistema experto** que asesore a un usuario en la gestión económica de una cafetería de especialidad ubicada en la Zona Metropolitana de Guadalajara (ZMG), empleando como interfaz un bot de Discord con componentes interactivos.

Objetivos específicos

- Modelar los costos fijos y variables de una cafetería real en GDL con precios en MXN.
- Implementar un motor de reglas IF-THEN que genere inferencias explicables sobre el estado del negocio.
- Diseñar tres agentes con responsabilidades diferenciadas: Atención, Pedido/Inferencia y Supervisor.
- Simular eventos macroeconómicos (heladas, tendencias virales, inflación, competencia) que afecten la demanda y los costos.
- Proveer retroalimentación en lenguaje natural al usuario a través del Agente Supervisor.

Descripción del Sistema

TAI (“Tutor de Análisis de Ingresos”) es un bot de Discord que simula la operación mensual de una cafetería de especialidad en Guadalajara. El usuario interactúa mediante *slash commands* y botones; el sistema responde con análisis económico generado por el motor de inferencias.

Dominio de conocimiento

El dominio es la **gestión económica** de una cafetería en la ZMG: costos de renta por zona, nómina de baristas, insumos (grano, leche, vasos), merma, elasticidad-precio de la demanda y eventos de mercado.

Flujo general

1. El usuario ejecuta un comando (p.ej. `/ciclo_avanzar`).
2. El **Agente 1** interpreta la intención.
3. El **Agente 2** inyecta un evento global aleatorio, corre el motor de reglas, resuelve el ciclo económico y persiste el resultado en la base de datos.
4. El **Agente 3** lee el registro de inferencias y genera una explicación en lenguaje natural.
5. Discord muestra el resultado como un *embed* con color condicional (verde/amarillo/rojo según el flujo neto).

Arquitectura del Sistema

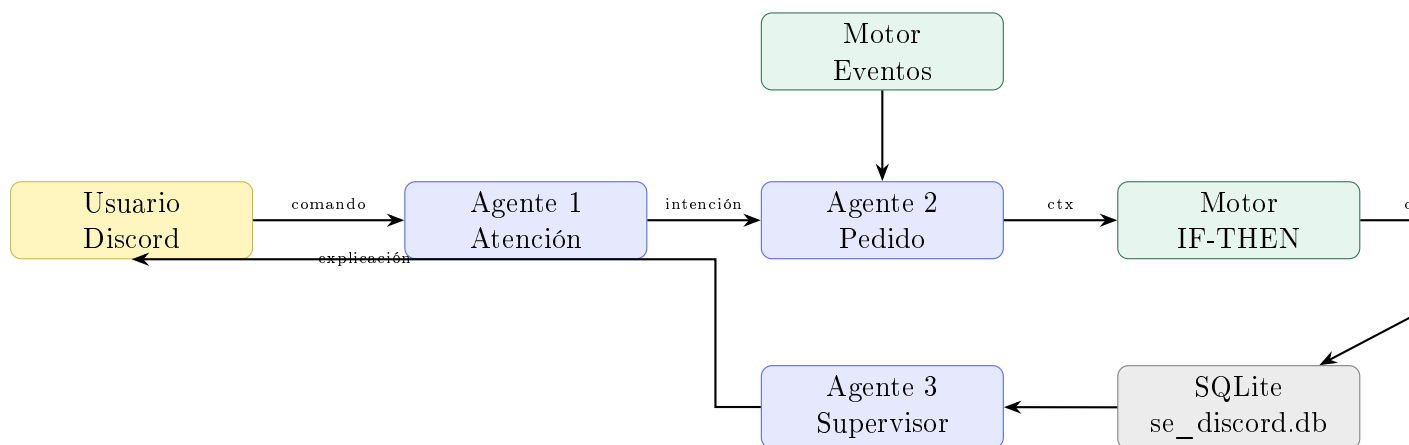
Estructura de archivos

```

1 SE-Discord/
2 |-- bot.py                # Entry point
3 |-- core/
4 |   |-- bd.py             # DB access (SQLite, no ORM)
5 |   |-- agentes.py        # Los 3 agentes
6 |   |-- economia.py       # Formulas economicas
7 |   |-- reglas.py         # Motor IF-THEN
8 |   |-- eventos.py        # Motor de eventos
9 |-- cogs/
10 |   |-- simulacion.py     # Slash commands + UI
11 |   |-- dashboard.py     # /dashboard
12 +-- docs/
13 |   |-- GG_registro_Proj.pdf
14 |   |-- manual.pdf

```

Diagrama de arquitectura



Componentes principales

Módulo	Responsabilidad
core/bd.py	Todas las operaciones con la BD (CRUD sin ORM)
core/agentes.py	Definición y lógica de los 3 agentes
core/economia.py	Fórmulas puras: demanda, ciclo, flujo neto
core/reglas.py	Motor IF-THEN; lista REGLAS evaluada en orden
core/eventos.py	Catálogo de 6 eventos con pesos aleatorios
cogs/simulacion.py	Slash commands + Buttons + Select Menus
cogs/dashboard.py	Embed de estado financiero con color condicional

Base de Conocimiento

La base de conocimiento está formada por tres capas de hechos y reglas.

4.1 Parámetros económicos (hechos)

Concepto	Descripción	Valor MXN
Renta base	Por ciclo mensual, zona Centro	\$18,000
Nómina	Barista Jr. por ciclo	\$8,500
Servicios	CFE + agua + gas	\$4,500
Mantenimiento	Máquina de espresso	\$1,500
Grano comercial	Costo variable/taza	\$6.00
Grano especialidad	Chiapas/Veracruz/Nayarit	\$14.00
Leche entera	Costo variable/taza	\$4.00
Alternativa vegetal	Avena/almendra	\$9.00
Vaso/empaque	Desechable	\$3.00
Merma	Sobre el insumo consumido	7.5 %
Precio mercado	Referencia GDL especialidad	\$55.00
Capacidad	Tazas máximas por ciclo	700

4.2 Zonas de la ZMG

Zona	Mult. Renta	Mult. Demanda
Centro Histórico GDL	1.0	1.1
Col. Americana / Chapultepec	1.4	1.3
Providencia	1.5	1.2
Andares / Puerta de Hierro	1.8	1.1
Tlaquepaque / Tonalá	1.1	1.0
Periferia / Colonia popular	0.6	0.8

4.3 Eventos macroeconómicos

Tipo	Descripción	Impacto
clima	Helada en Brasil	$\text{costo_grano} \times 1.25$
tendencia	Viral en TikTok	$\text{demanda} \times 1.40$
inflacion	Inflación local	$\text{costos_fijos} \times 1.05$
competencia	Abre cafetería rival	$\text{demanda} \times 0.85$
cosecha	Excelente cosecha Colombia	$\text{costo_grano} \times 0.90$
estable	Mes tranquilo	sin cambios (peso $3\times$)

Inferencias Utilizadas

El motor evalúa cuatro reglas en el orden declarado. Cada regla que se dispara genera un registro en la tabla `registro_inferencias` con entrada, resultado y explicación.

Regla 1 — Precio alto

IF `precio_usuario > precio_mercado`
THEN “Demanda cae exponencialmente”

Entrada: precio actual y precio de referencia (\$55)

Resultado: porcentaje de sobreprecio informado al usuario

Explicación: cobrar más que el mercado reduce la afluencia según el modelo de elasticidad (exponente 1.5) salvo que se compense con calidad o marketing.

Regla 2 — Stock insuficiente

IF inventario_tazas < demanda_estimada

THEN “Faltan $\approx N$ tazas de insumo”

Efecto: {sugerencia: "reabastecer"}

Explicación: la demanda supera el inventario; se pierden ventas por desabasto. Se recomienda usar /proveedor antes del próximo ciclo.

Regla 3 — Grano de especialidad

IF tipo_grano == "especialidad"

THEN “Reputación sube”

Efecto: {reputacion_mult: 1.15}

Explicación: usar grano de origen (Chiapas/Veracruz/Nayarit) mejora la percepción de calidad y eleva la demanda base en un 15 %.

Regla 4 — Riesgo de bancarrota

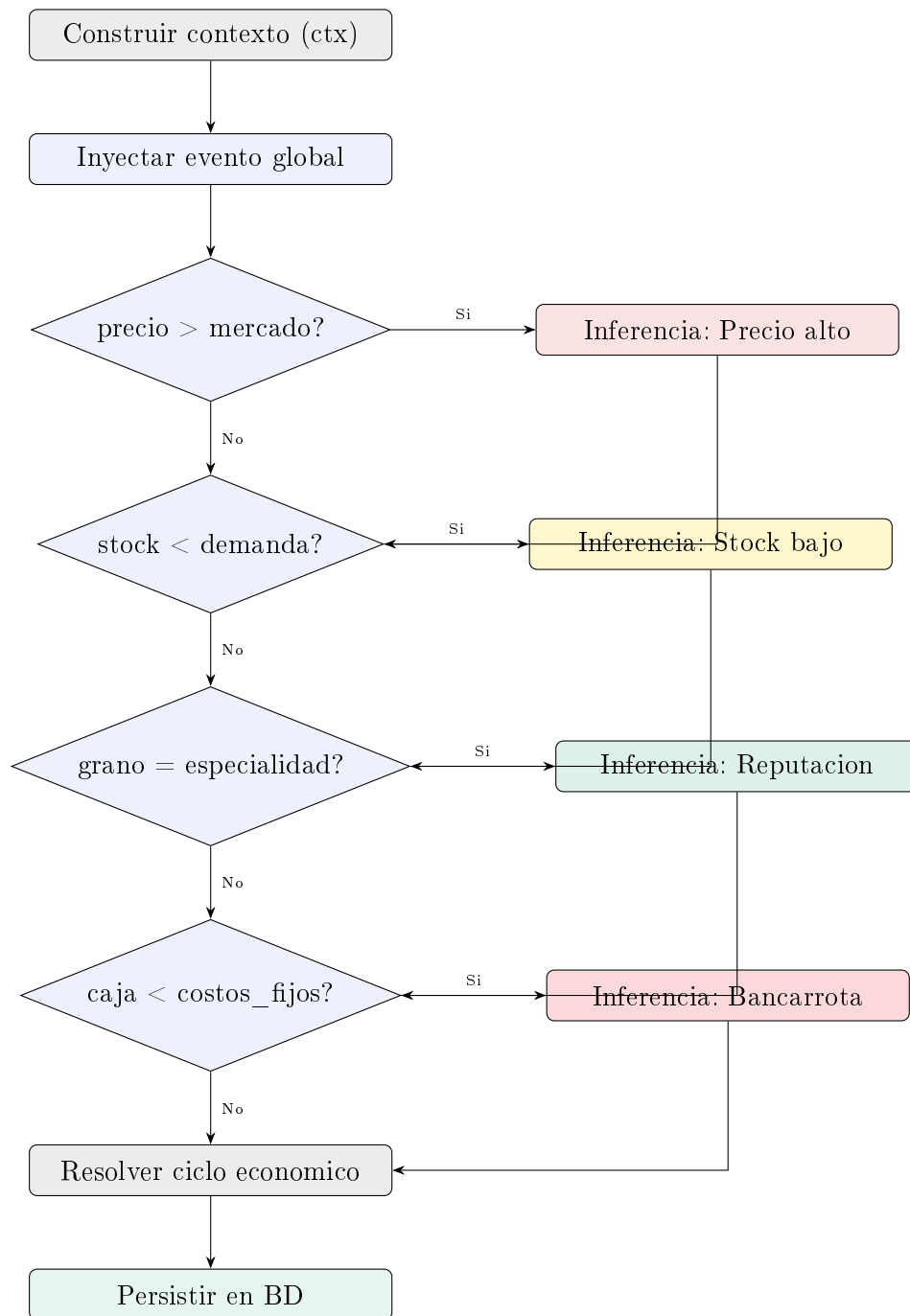
IF caja < costos_fijos

THEN “ALERTA: riesgo de no cubrir costos fijos”

Efecto: {alerta: "bancarrota"}

Explicación: la caja disponible no alcanza para cubrir los costos fijos del ciclo. El negocio puede quebrar si no se incrementan ingresos o se reducen gastos.

Diagrama del motor de inferencias



Explicación de los Agentes

Agente 1 — Atención (AgenteAtencion)

Traduce el texto o comando del usuario en una estructura `Intencion(accion, parametros)`.

```

1 class AgenteAtencion:
2     def interpretar(self, texto: str) -> Intencion:
3         t = texto.lower().strip()
4         if any(p in t for p in ("avanzar", "ciclo", "mes")):
5             return Intencion("avanzar_ciclo", {})
  
```

```

6         if any(p in t for p in ("comprar", "reabastecer")):
7             return Intencion("comprar", {"item": "grano"})
8         if "precio" in t:
9             return Intencion("ajustar_precio", {})
10        return Intencion("desconocida", {})

```

Responsabilidad: es el único punto de contacto con el canal de Discord. Desacopla la capa de presentación de la lógica de negocio.

Agente 2 — Pedido / Inferencia (AgentePedido)

Es el núcleo del sistema experto. Sigue el pipeline:

1. Obtiene el inventario y calcula el costo variable real.
2. Genera un evento macroeconómico aleatorio.
3. Aplica el impacto del evento al contexto.
4. Calcula la demanda con elasticidad-precio.
5. Evalúa las reglas IF-THEN (motor de inferencias).
6. Resuelve el ciclo económico (unidades, ingresos, flujo neto).
7. Persiste finanzas, evento e inferencias en SQLite.
8. Consume inventario con merma del 7.5 %.
9. Aplica decaimiento del bonus de marketing ($\times 0.5$).

Método principal: `procesar_ciclo(negocio) -> ResultadoAgentePedido`

Agente 3 — Supervisor (AgenteSupervisor)

Lee la tabla `registro_inferencias` y genera una explicación en lenguaje natural para el usuario.

```

1 def explicar(self, negocio_id: int, ciclo: int) -> str:
2     # Lee todas las inferencias del ciclo actual
3     # Formatea: "- [resultado]\n -> [explicacion]"
4     # Si no hubo reglas: "Operacion normal."

```

Responsabilidad: garantiza la *explicabilidad* del sistema. El usuario puede siempre saber *por qué* el sistema tomó una decisión.

Agente	Entrada	Salida
Atención	Texto / slash command	<code>Intencion</code>
Pedido	<code>Intencion</code> + BD	<code>ResultadoAgentePedido</code>
Supervisor	<code>negocio_id</code> , <code>ciclo</code>	<code>str</code> (lenguaje natural)

Base de Datos

Se usa **SQLite 3** con acceso directo mediante `sqlite3` de la biblioteca estándar de Python (sin ORM). El archivo `se_discord.db` se crea automáticamente en la raíz del proyecto.

Tablas

Tabla	Descripción
negocios	Datos maestros del negocio (caja, precio, zona, marketing)
inventario	Tazas equivalentes por tipo de insumo
finanzas_registro	Histórico de ciclos (ingresos, costos, flujo)
historial_eventos	Evento global disparado por ciclo
registro_inferencias	Columna vertebral de la explicabilidad

Diagrama ER simplificado

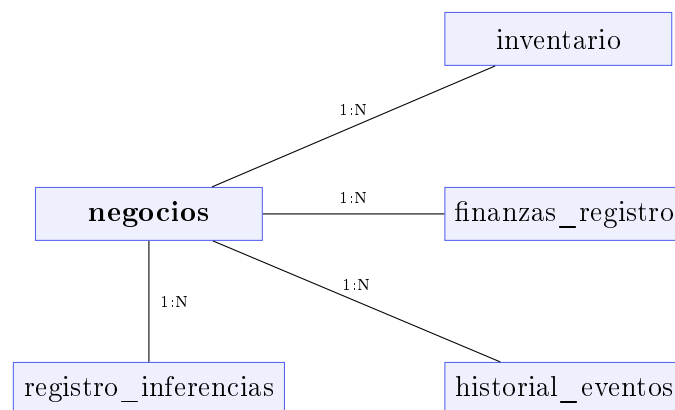


Tabla registro_inferencias (explicabilidad)

Columna	Contenido
negocio_id	FK a negocios
ciclo	Número del ciclo en que se disparó
agente	"pedido" / "atencion"
regla	Texto de la regla IF-THEN
entrada	Valores que activaron la regla
resultado	Decisión tomada
explicacion	Justificación en lenguaje natural

Herramientas Utilizadas

Herramienta	Versión	Uso
Python	3.11+	Lenguaje principal
discord.py	2.x	Framework del bot y UI interactiva
SQLite3	(stdlib)	Base de datos embebida
LaTeX / MiKTeX	2024	Documentación técnica y manual
Git / GitHub	2.x	Control de versiones y entrega
Discord Developer Portal	–	Registro del bot y permisos
Figma	–	Prototipo de interfaz (https://gown-kinder-24829009.figma)

Fórmulas económicas implementadas

$$\text{Margen de contribución} = \text{precio} - \text{costo_variable_unitario}$$

$$\text{Punto de equilibrio} = \frac{\text{costos_fijos}}{\text{margen_contribución}}$$

$$\text{Demanda} = \text{demanda_base} \times \left(\frac{\text{precio_mercado}}{\text{precio_usuario}} \right)^{1.5}$$

$$\text{Unidades vendidas} = \min(\text{demanda}, \text{capacidad}, \text{inventario})$$

$$\text{Flujo neto} = \text{ingresos} - (\text{costos_fijos} + \text{costos_variables})$$

Conclusiones

El proyecto **TAI** demuestra que es viable construir un sistema experto funcional sobre una plataforma de mensajería como Discord, aprovechando sus componentes interactivos (buttons, select menus, embeds) como interfaz natural para un usuario no técnico.

- La separación en tres agentes con responsabilidades claras facilita el mantenimiento y la extensión del sistema.
- El módulo `registro_inferencias` hace que el sistema sea **explicable**: el usuario siempre puede saber por qué se tomó cada decisión económica.
- Las fórmulas puras en `core/economia.py` son independientes de la capa de persistencia, lo que permite probarlas de forma aislada.
- El modelo económico con zonas, tipos de insumo y eventos macroeconómicos refleja la realidad de operar una cafetería en Guadalajara.
- Discord.py facilita el prototipado rápido de interfaces conversacionales con componentes gráficos sin necesidad de un frontend web.

Manual de Usuario

11.1 Requisitos previos

- **Python 3.11 o superior** instalado en el equipo.
- **Cuenta de Discord** y acceso a un servidor donde agregar el bot.
- **Token de bot de Discord** generado en el Developer Portal.

11.2 Obtener el token del bot

1. Visita <https://discord.com/developers/applications>.
2. Crea una nueva aplicación o selecciona la existente.
3. En el menú lateral, ve a **Bot**.
4. Activa **Message Content Intent** en “Privileged Gateway Intents”.
5. Copia el token con el botón **Reset Token**.

11.3 Instalación

```
1 # 1. Clona el repositorio
2 git clone https://github.com/Ortiz-Ascencio-Alvaro-Martin/SE-
   Discord.git
3 cd SE-Discord
4
5 # 2. (Recomendado) Crea un entorno virtual
6 python -m venv .venv
7
8 # Windows:
9 .venv\Scripts\activate
10 # Linux / macOS:
11 source .venv/bin/activate
12
13 # 3. Instala dependencias
14 pip install -r requirements.txt
```

El archivo `requirements.txt` contiene:

```
1 discord.py>=2.3.0
```

11.4 Configuración del token

```
1 # Windows (PowerShell)
2 $env:DISCORD_TOKEN = "tu_token_aqui"
3
4 # Linux / macOS
5 export DISCORD_TOKEN="tu_token_aqui"
```

Importante: nunca subas el token al repositorio.

11.5 Ejecutar el bot

```
1 python bot.py
```

Si todo está correcto verás en la consola:

```
1 Conectado como TAI#7626 - comandos sincronizados
```

La primera vez se crea automáticamente `se_discord.db` con las cinco tablas del esquema.

11.6 Invitar el bot a tu servidor

Usa el enlace oficial de instalación:

https://discord.com/oauth2/authorize?client_id=1511180179395838083

Abre el enlace en el navegador, selecciona tu servidor y acepta los permisos solicitados.

11.7 Guía de uso — Comandos

/crear_negocio

Crea tu cafetería. Debes proporcionar:

- **nombre** — nombre de tu cafetería.
- **zona** — elige una de las 6 zonas de la ZMG del selector.

El sistema asigna caja inicial de \$100,000 MXN, precio de \$55/taza e inventario inicial de 100 tazas (grano comercial) + 60 tazas (leche).

/dashboard

Muestra el estado completo:

- Caja actual, precio por taza y bonus de marketing activo.
- Inventario desglosado por tipo de insumo.
- Resultados del último ciclo (ingresos, costos, flujo neto).
- Indicadores: margen de contribución, punto de equilibrio, costo variable unitario y costos fijos del ciclo.

Si hay más de una cafetería en el servidor, aparece un selector para elegir cuál ver.

/ciclo_avanzar

Simula un mes fiscal completo:

1. Se genera un evento macroeconómico aleatorio.
2. El motor de reglas evalúa el estado del negocio.
3. Se calcula demanda, ventas, costos y flujo neto.
4. Se persisten resultados en la base de datos.
5. Se consume el inventario (con merma del 7.5 %).
6. El bonus de marketing decae al 50 %.

Aparecen cuatro botones:

- **Avanzar ciclo** — repite el ciclo sin escribir el comando.
- **Ver explicación** — muestra el reporte del Agente Supervisor.
- **Vender** — venta rápida proporcional a la demanda actual.
- **Pagar Nómina** — descuenta \$8,500 MXN de la caja manualmente.

/proveedor

Abre un Select Menu para comprar insumos:

Insumo	Calidad	Costo	Tazas
Grano comercial	Media	\$300	+50
Grano especialidad	Alta (Chiapas/Veracruz)	\$700	+50
Leche entera	Base	\$120	+30
Alternativa vegetal	Avena/almendra	\$270	+30

/marketing

Lanza una campaña que eleva la demanda base temporalmente:

Campaña	Bonus demanda	Costo	Decaimiento
Redes sociales	+10 %	\$3,000	50 %/ciclo
Influencer	+20 %	\$8,000	50 %/ciclo
Descuento apertura	+25 %	\$1,500	50 %/ciclo
Cata en tienda	+15 %	\$5,000	50 %/ciclo

/ajustar_precio

Cambia el precio por taza. El sistema advierte si el precio está más de \$10 por encima o por debajo del precio de mercado (\$55 MXN).

`/borrar_negocio`

Elimina permanentemente la cafetería con todos sus datos (inventario, historial, finanzas e inferencias). Requiere confirmación con un botón.

11.8 Interpretar el Dashboard

- **Color del embed:** verde = flujo positivo, amarillo = pérdida leve ($< \$5,000$), rojo = pérdida mayor, gris azul = sin ciclos.
- **Margen de contribución:** si es negativo, cada taza vendida genera pérdida. Sube el precio o baja el costo de insumo.
- **Punto de equilibrio:** número mínimo de tazas que debes vender para no perder dinero. Compara con las tazas vendidas del último ciclo.

11.9 Estrategias recomendadas

1. **Inicio:** compra grano comercial + leche. Precio \$55. Avanza ciclos.
2. **Crecimiento:** cuando la caja supere \$30,000, compra grano de especialidad para activar el bono de reputación (+15 % demanda).
3. **Marketing:** con caja sólida, lanza “Descuento apertura” (\$1,500, +25 %) para maximizar ventas un ciclo.
4. **Ajuste de precio:** si la regla “Precio alto” se dispara repetidamente, baja el precio o compensa con calidad/marketing.
5. **Zona premium:** Andares ofrece la mayor renta ($\times 1.8$) pero también demanda alta ($\times 1.1$); es rentable solo con precio y calidad altos.

11.10 Preguntas frecuentes

El bot no responde a los comandos slash. Asegúrate de que el bot tenga el permiso `applications.commands` y de que los slash commands se hayan sincronizado (`on_ready` llama a `bot.tree.sync()`).

La caja se fue a negativo. El simulador permite saldo negativo para reflejar deuda real. Usa `/proveedor` para reabastecerte y avanza ciclos con mayor precio.

¿Cómo reiniciar mi cafetería? Usa `/borrar_negocio` y luego `/crear_negocio` con los nuevos parámetros.

¿Qué pasa si el inventario llega a cero? El ciclo venderá 0 tazas. El flujo neto será negativo por los costos fijos. Usa `/proveedor` antes de avanzar el siguiente ciclo.